

InvenIQ

AI-Integrated Inventory Intelligence Platform

JAIN Online University

Master of Computer Applications (MCA)

Project Synopsis - Academic Year 2025-2026

Project Type:	AI-Powered Full-Stack Web Application
Tech Stack:	React 18 + FastAPI + MySQL + GPT-4o
Modules Covered:	4 Core Modules (from a larger enterprise system)
Submission Date:	8 June 2026
Status:	Ready for Academic Submission

CONFIDENTIAL - FOR ACADEMIC EVALUATION ONLY

This document contains selectively disclosed modules. Full system details are proprietary.

Abstract

Traditional inventory management systems suffer from reactive decision-making, fragmented data flows, and complete absence of predictive capability. This project presents InvenIQ, an enterprise-grade, AI-augmented inventory intelligence platform built on a modern full-stack architecture (React 18 + FastAPI + MySQL + GPT-4o). The system replaces spreadsheet-driven workflows with real-time dashboards, automated procurement pipelines, intelligent demand visibility, and a conversational AI business assistant - enabling businesses to make data-driven decisions with minimal manual intervention at every stage of their inventory lifecycle.

Problem Statement

- No centralized real-time visibility into stock levels, inward movements, or outward dispatch.
- Purchase orders tracked manually in spreadsheets - prone to errors and costly delays.
- Sales data and customer records siloed across tools with no unified operational view.
- Zero forecasting capability leading to chronic overstocking or stockout situations.
- No actionable insight layer - management reports generated manually, often days late.

Objectives

1. Design a scalable full-stack web platform for end-to-end inventory lifecycle management.
2. Implement an automated procurement workflow: Purchase Requisition -> Purchase Order -> GRN.
3. Build a sales and customer management system with order tracking and credit monitoring.
4. Integrate GPT-4o as a conversational business intelligence engine with live data tools.
5. Deliver a production-grade, role-based, JWT-authenticated multi-user system.

Technology Stack

Layer	Technology
Frontend	React 18, Chart.js, CSS3 token-based design system
Backend	FastAPI (Python 3.11), asyncio, aiomysql
Database	MySQL 8.0
AI Engine	OpenAI GPT-4o via Async REST API
Auth	JWT (access + refresh tokens), bcrypt password hashing
Deployment	Localhost / VPS - environment-variable driven configuration
Dev Tools	VS Code, Postman, Git

Project Scope - 4 Selected Modules

Module 1 - Inventory Management & Stock Tracking

Purpose

Provides real-time visibility into current stock, inward receipts, movement history, and dead-stock identification across all SKUs.

Key Features

- SKU-level stock ledger with movement type classification: GRN Receipt, Sale Dispatch, Purchase Return, Damage Write-off.
- Inward register - records every goods receipt against a supplier with quantity, batch, and date.
- Dead-stock detection engine - flags SKUs with zero movement beyond a configurable threshold (default: 90 days).
- KPI dashboard: Total SKUs | Total Stock Value | Items Below Reorder | Dead-Stock Count.
- Period-based filtering (Today / MTD / QTD / YTD) with CSV export for reconciliation.

Technical Highlights

- Async FastAPI endpoint using `asyncio.gather()` for parallel database queries.
- DB-first / demo-fallback pattern - system remains fully functional without MySQL.
- Chart.js area chart rendering stock movement trend over rolling periods.

Academic Relevance

Database normalization, indexed query optimization, React state management, and REST API design principles (resource-based routing, proper HTTP status codes, pagination).

Module 2 - Procurement Management (PR -> PO -> GRN)

Purpose

End-to-end procure-to-pay workflow from internal material requests through vendor ordering to goods receipt and quality verification.

Key Features

- Purchase Requisition: Department-wise requests with priority levels (CRITICAL / HIGH / MEDIUM / LOW); status lifecycle PENDING -> APPROVED -> CONVERTED; inline edit and duplicate capability.
- Purchase Order: PR-to-PO conversion with supplier selection; conflict prevention (one DRAFT PO per supplier per PR); auto-progression DRAFT -> PARTIAL -> RECEIVED -> FULLY_RECEIVED.
- Goods Receipt Note (GRN): Quantity verification against PO line items; over-receiving prevention (422 error); journal entry auto-generated (DR Stock / CR Accounts Payable).
- Quality Control: 4-quantity decision form per line (Accepted / Rejected / Rework / Hold); QC completion gate enforced before invoice matching proceeds.

Technical Highlights

- Multi-table transactional writes with rollback safety using `aiomysql`.
- Status machine validation at API layer - business rules enforced server-side.
- Audit log service records every status change with user identity and timestamp.
- Idempotent schema migrations executed automatically on application boot.

Academic Relevance

Enterprise workflow state machines, transactional database integrity, service-layer architecture, and audit trail design patterns.

Module 3 - Sales & Customer Management

Purpose

Centralized tracking of customer accounts, sales orders, and outstanding credit positions across the entire business.

Key Features

- Customer master with contact details, GST number, city, and account status.
- Sales order register at line-item level: product, quantity, rate, and dispatch status.
- Outstanding order dashboard - pending vs. dispatched vs. invoiced breakdown.
- Customer-level credit exposure view: credit limit, utilized amount, overdue flag.
- Sales trend chart - monthly revenue over rolling 12 months (Chart.js gradient-fill line chart).
- Server-side paginated API for scalable table rendering across large datasets.
- Period selector (Today / MTD / QTD / YTD) propagated across all data-fetching views.

Technical Highlights

- Paginated API endpoint (limit / offset) preventing full-table scans on large datasets.
- React state coordination between search filter and pagination components simultaneously.
- Silent background auto-refresh on window focus without disrupting the user experience.

Academic Relevance

Pagination design trade-offs (client-side vs. server-side), component-level state isolation in React, and relational data modeling for CRM-adjacent business systems.

Module 4 - AI-Powered Business Intelligence Assistant

Purpose

A conversational GPT-4o assistant embedded in the platform that answers operational queries, generates proactive insights, and surfaces actionable intelligence from live business data - replacing manual report generation entirely.

4-Path Intelligent Routing Architecture

Path	Routing Behavior
Path 1	Generic / greeting queries -> direct conversational response
Path 2	Knowledge queries -> structured KB (inventory, GST, procurement rules)
Path 3	Insight queries -> full-platform data aggregation -> proactive KPI analysis
Path 4	Operational queries -> tool selector -> live DB fetch -> GPT-4o response

Key Features

- Natural language query interface with streaming token output via SSE (Server-Sent Events).
- Live data tools covering stock levels, demand trends, supplier performance, customer outstanding.
- Proactive anomaly detection: stockouts, demand spikes, overdue payments without user prompting.
- Root-cause analysis templates (PDCA / 5-Why / Fishbone) auto-applied to operational issues.
- Create Purchase Order via chat using GPT-4o function calling -> confirmation card -> DB write.

Technical Highlights

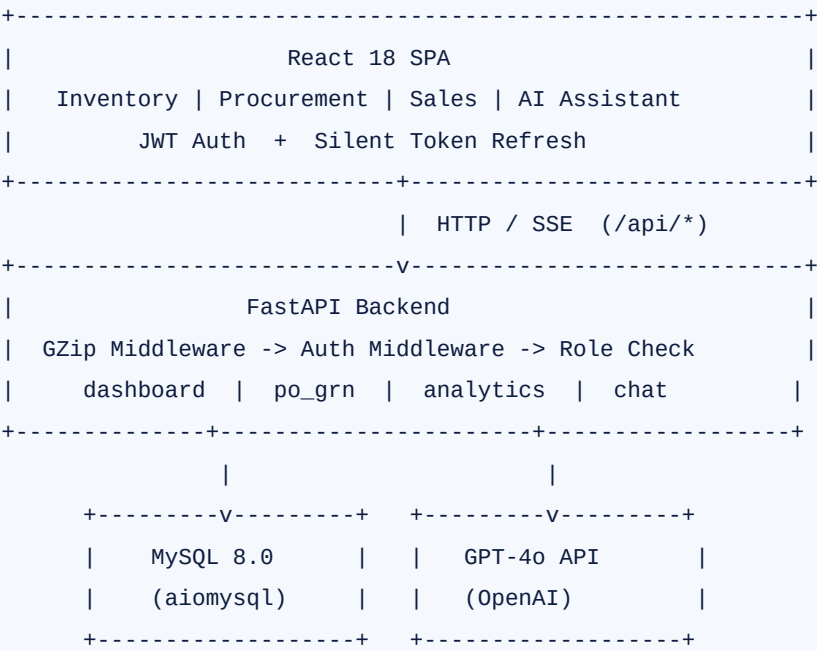
- OpenAI AsyncOpenAI client with 60-second timeout and graceful API failure degradation.
- SSE streaming delivers first token in under 1.5 seconds for perceived performance.
- Client disconnect detection stops LLM generation on tab close (eliminates wasted API cost).
- `asyncio.gather()` runs all data tools in parallel during insight generation.

Academic Relevance

LLM integration architecture, SSE streaming protocols, tool-use / function calling, multi-path AI routing design, and safety-first

fallback pattern implementation.

System Architecture



Authentication & Role-Based Access Control

Feature	Detail
Access Token	JWT, 8-hour expiry, signed with HS256
Refresh Token	7-day expiry, DB-backed rotation on each use
Password Hashing	bcrypt with salt rounds
Role Hierarchy	Admin > Sales Manager > Warehouse Manager > CFO
Module Access	Enforced at API middleware layer per JWT claim
Public Routes	/api/health and /api/auth/* only

Expected Outcomes

Outcome	Target Metric
Stock Visibility	Real-time SKU-level ledger with full movement history
Procurement Automation	PR to GRN completed in under 3 screen interactions
Sales Tracking	Order-level dispatch and outstanding visibility
AI Response Latency	First streaming token delivered in under 1.5 seconds
Offline Demo Mode	Fully functional without any database connection

Limitations (Academic Scope)

- Multi-warehouse distributed inventory is not in scope for this phase.
- Payment gateway integration is not included.
- Mobile application is out of scope - responsive web interface only.

ERP integration (SAP, Tally) is not covered in this submission.

References

1. FastAPI Official Documentation <https://fastapi.tiangolo.com>
2. React 18 Documentation <https://react.dev>
3. OpenAI API Reference <https://platform.openai.com/docs>
4. MySQL 8.0 Reference Manual <https://dev.mysql.com/doc>
5. OWASP Top 10 Web Application Risks <https://owasp.org>
6. JSON Web Tokens RFC 7519 <https://tools.ietf.org/html/rfc7519>
7. Chart.js Documentation <https://www.chartjs.org/docs>
8. aiomysql Documentation <https://aiomysql.readthedocs.io>